实验四 决策树与随机森林分类

一、 实验目的

通过实验更好地掌握机器学习及优化的相关概念、技术、原理、应用等；通过实验提高学生编写实验报告、总结实验结果的能力；使学生对机器学习模型、算法等有比较深入的认识。需掌握的知识点如下：

掌握机器学习中涉及的相关概念、模型、算法；

熟悉机器学习模型训练、验证、测试的流程；

熟悉常用的数据预处理方法；

掌握决策树、随机森林的表示、求解及编程。

二、基本要求

准备好实验数据，编程完成实验内容，收集实验结果。

独立完成实验报告。

三、实验软件

推荐使用 Python 编程语言（允许使用 numpy 库，需实现详细实验步骤，不允许直接调用 scikit-learn 中回归、分类等高层 API）。

四、实验内容

基于 Adult 数据集，完成关于收入是否大于 50K 的贪心决策树分类、随机森林分类模型训练、测试与评估。

1）准备数据集并认识数据

下载 Adult 数据集

http://archive.ics.uci.edu/ml/datasets/Adult

了解数据集各个维度特征及预测值的含义。

说明：Adult 数据集，又称“人口普查收入数据集”（Census Income Dataset），是机器学习领域的经典基准数据集。其核心价值在于通过丰富的个体特征预测年收入是否超过 50,000 美元。

2）探索数据并预处理数据

观察数据集各个维度特征及预测值的数值类型与分布。

预处理各维度特征，参考：https://blog.csdn.net/SanyHo/article/details/105304292

3）训练模型

编程实现训练数据集上贪心决策树、随机森林的构建。

4）测试和评估模型

在测试数据集上计算所训练模型的准确率、AUC 等指标。

五、学生实验报告

（1）简要介绍贪心决策树、随机森林分类的原理

贪心决策树原理：

①决策树的基本思想

决策树是基于特征对实例（sample）进行分类模型，可以理解为给定特征条件下类的条件概率

分布。要进行分类的样本即给定的特征值，要预测出的 label，即在给定特征值条件类的概率最大——所属的类。决策树此时充当划分特征空间的一种方式。特征空间的维数为数据集的特征个数。经过 ID3 或 C45 算法将特征空间进行划分，并且划分后的每个特征空间区间对应着发生概率最大的类别 label。属于 count-based 型。决策树的叶结点表示数据集中的类 label，内部结点表示选择划分的特征。

②一些概念

熵：随机变量不确定性的度量，熵越大，不确定性越高。

设 X 是一个取有限个值的离散随机变量，其概率分布为：

$$P(X=x_i)=p_i, i=1,2,\ldots,n$$

则随机变量 X 的熵定义为

$$H(X)=-\sum_{i=1}^{n} p_i \log p_i$$

条件熵：表示已知随机变量 X 的条件下随机变量 Y 的不确定性——$H(Y|X)$。

$$H(Y|X)=\sum_{x\in X} p(x) H(Y|X=x)$$

其中 $p(x)$ 表示 $X=x$ 发生的概率。

在 ID3 算法下一般 Y 表示的都是类。

信息增益：一般为特征选择的方式常用的还有卡方检验，交叉熵，信息增益——$G(D,A)$ 表示由选择特征 A 而使得对数据集分类的不确定性减少的程度，减少的越多，数据集分类的不确定性越低。表示特征 A 对数据集 D 分类影响效果越好。

$$G(D,A)=H(D)-H(D|A)$$

其中 $H(D)$ 表示数据集 label 类别的熵即每个 label 取不同类别的值的时候的不确定性。其中 $H(D|A)$ 表示在选择特征 A 的条件下，数据集 label 类别的熵。此时也可以表示类别 label 与特征的互信息在决策树中公式的应用。

③决策树算法的主要步骤

特征选择：需要根据数据集的特点进行选择，有信息增益、信息增益比、Gini 指数。即如何划分特征空间。本实验中实现 ID3 算法，即使用信息增益来进行特征选择。

**决策树的生成：根据特征选择的算法，对数据集进行递归的生成树。

**决策树的修剪：由于递归生成的决策树对训练数据分类准确，但对未知的测试数据却不再那么准确，也就是泛化能力较弱，过于拟合训练数据。此时需要对生成的复杂决策树进行简化处理，从而让拟合状态变成正常化状态。

②随机森林分类原理：

定义：随机森林指的是利用多棵决策树对样本进行训练并预测的一种分类器。可回归可分类。

随机森林是基于多棵决策树的一种集成学习算法，常见的决策树算法主要有以下几种：

ID3：使用信息增益 $G(D,A)$ 进行特征选择

C4.5：信息增益率 $=G(D,A)/H(A)$

CART：基尼系数

一个特征的信息增益（或信息增益率，或基尼系数）越大，表明特征对样本的熵的减少能力更强，这个特征使得数据由不确定性到确定性的能力越强。本实验采用 ID3 算法。

随机森林与决策树的不同

简单来说，随机森林就是对决策树的集成，但有两点不同：

（1）采样的差异性：从含 m 个样本的数据集中有放回的采样，得到含 m 个样本的采样集，用于训练。这样能保证每个决策树的训练样本不完全一样。

（2）特征选取的差异性：每个决策树的 n 个分类特征是在所有特征中随机选择的（n 是一个需要我们自己调整的参数）

步骤

1）数据的随机选取：

首先，从原始的数据集中采取有放回的抽样，构造子数据集，子数据集的数据量是和原始数据集相同。不同子数据集的元素可以重复，同一个子数据集中的元素也可以重复。

2）待选特征的随机选取

与数据集的随机选取类似，随机森林中的子树的每一个分裂过程并未用到所有的待选特征，而是从所有的待选特征中随机选取一定的特征，之后再在随机选取的特征中选取最优的特征。这样能够使得随机森林中的决策树都能够彼此不同，提升系统的多样性，从而提升分类性能。

3）决策结果

利用子数据集以及随机选取的特征来构建子决策树，多个子决策树组成随机森林。将测试数据放到每个子决策树中，每个子决策树输出一个结果，投票决定最终分类。

（2）程序清单（包含详细求解步骤）

①导入相应库支持

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from math import log
import operator
import random
from random import randrange
from collections import Counter
```

②导入数据集，观察数据特点

```
#导入数据集
train_set=pd.read_csv( filepath_or_buffer: 'adult/adult.data',header=None,names=['age','workclass','fnlwgt','education',
                                                                                  'education-num','marital-status',
                                                                                  'occupation','relationship','race',
                                                                                  'sex','capital-gain','capital-loss',
                                                                                  'hours-per-week','native-country','income'])
train_set.info()
print(train_set.describe())
test_set=pd.read_csv( filepath_or_buffer: 'adult/adult.test',header=None,names=['age','workclass','fnlwgt','education',
                                                                                 'education-num','marital-status',
                                                                                 'occupation','relationship','race',
                                                                                 'sex','capital-gain','capital-loss',
                                                                                 'hours-per-week','native-country','income'])
test_set.info()
# print(test_set)
```

其中

```
train_set.info()
```

```
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             32561 non-null  int64
 1   workclass       32561 non-null  object
 2   fnlwgt          32561 non-null  int64
 3   education       32561 non-null  object
 4   education-num   32561 non-null  int64
 5   marital-status  32561 non-null  object
 6   occupation      32561 non-null  object
 7   relationship    32561 non-null  object
 8   race            32561 non-null  object
 9   sex             32561 non-null  object
 10  capital-gain    32561 non-null  int64
 11  capital-loss    32561 non-null  int64
 12  hours-per-week  32561 non-null  int64
 13  native-country  32561 non-null  object
 14  income          32561 non-null  object
dtypes: int64(6), object(9)
```

③进行数据预处理

1）处理缺失值

```
#处理缺失值 将含有值' ? '的数据行去掉  注意问号前有一个空格，处理缺失值后数据的index变成不是连续的
for i in train_set.columns:
    test_set=test_set[test_set[i]!=' ?']
    train_set=train_set[train_set[i]!=' ?']

#重新规划数据集的index
test_set.index=range(0,len(test_set))
train_set.index=range(0,len(train_set))
# print(train_set)
# print(test_set)
```

2）删除 fnlwgt 列（即序号列）该列不影响分类结果

```
#删除fnlgwt列,序号列
train_set.drop( labels: 'fnlwgt',axis=1,inplace=True)
test_set.drop( labels: 'fnlwgt',axis=1,inplace=True)
```

3）删除 Education 列

```
#Eductaion和EduNum特征相似，可以删除Education
train_set.drop( labels: ['education'],axis=1,inplace=True)
test_set.drop( labels: ['education'],axis=1,inplace=True)
```

4）将一些连续值处理为离散值，合并一些 object 类型的属性列值。观察各属性列值的特点后，做出以下处理

4.1 将 age 属性列值范围划分为 0-25,25-50,50-75,75-100

```
#1.将age属性划分为0-25  25-50   50-75   75-100
age=np.copy(train_set['age'])
age_test=np.copy(test_set['age'])

def transform1(age,size):  3 用法
    for i in range(size):
        if age[i]>=0 and age[i]<25:
            age[i]=0
        elif age[i]>=25 and age[i]<50:
            age[i]=1
        elif age[i]>=50 and age[i]<75:
            age[i]=2
        elif age[i]>=75:
            age[i]=3
    return age
age=transform1(age,m)
age_test=transform1(age_test,n)
train_set['age']=age
test_set['age']=age_test
```

4.2 处理 capital-gain 属性列，>0 的值用 1 代换，=0 的值用 0 代换

```
#2.将capital-gain属性    >0的值都用1替换   =0的值都用0替换
gain=np.copy(train_set['capital-gain'])
gain_test=np.copy(test_set['capital-gain'])

def transform2(gain,size):  3 用法
    for i in range(size):
        if gain[i]==0:
            gain[i]=0
        elif gain[i]>0:
            gain[i]=1
    return gain

gain=transform2(gain,m)
gain_test=transform2(gain_test,n)
train_set['capital-gain']=gain
test_set['capital-gain']=gain_test
```

4.3 将 captional-loss 属性划分为>0 =0 两类

```
#3.将captional-loss属性 划分为>0   =0两类
loss=np.copy(train_set['capital-loss'])
loss_test=np.copy(test_set['capital-loss'])


loss=transform2(loss,m)
loss_test=transform2(loss_test,n)
train_set['capital-loss']=loss
test_set['capital-loss']=loss_test
```

4.4 将 housr-per-week 划分为<40、==40、>40 等 3 类。

```python
#4.将housr-per-week划分为      <40    ==40    >40
hours=np.copy(train_set['hours-per-week'])
hours_test=np.copy(test_set['hours-per-week'])

def transform3(hours,size):  2 用法
    for i in range(size):
        if hours[i] < 40:
            hours[i] = 0
        elif hours[i] == 40:
            hours[i] = 1
        elif hours[i] >40:
            hours[i]=2
    return hours
hours=transform3(hours,m)
hours_test=transform3(hours_test,n)

train_set['hours-per-week']=hours
test_set['hours-per-week']=hours_test
```

4.5 将 Country 划分为 USA、not USA。

```python
#5.将Country划分为USA  not USA两类
cty=np.copy(train_set['native-country'])
cty_test=np.copy(test_set['native-country'])

def transform4(cty,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if cty[i] ==" United-States":
            cty[i] = 0
        elif cty[i] != " United-States":
            cty[i] = 1
    return cty
cty=transform4(cty,m)
cty_test=transform4(cty_test,n)

train_set['native-country']=cty
test_set['native-country']=cty_test
```

4.6 将 workclass 分为 Freelance、other、Proprietor、Private、Government 五类，其中 Freelance other Private 是本身就有的类，不做处理

```python
#6.将workclass分为Freelance   other   Proprietor   Private  Government五类
# print(train_set['workclass'].value_counts())
workclass=np.copy(train_set['workclass'])
workclass_test=np.copy(test_set['workclass'])
def transform6(workclass,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if workclass[i]==" Federal-gov" or workclass[i]== " Local-gov" or workclass[i]== " State-gov" :
            workclass[i] = " Government"
        elif workclass[i] == " Self-emp-not-inc" or  workclass[i]==" Self-emp-inc":
            workclass[i] = " Proprietor"
    return workclass
cty=transform6(workclass,m)
cty_test=transform6(workclass_test,n)

train_set['workclass']=workclass
test_set['workclass']=workclass_test
```

4.7 将 education-num 分为 0-5:0、5-10:1、>=10:2。

```python
#7.将education-num分为0-5:0   5-10:1  >=10:2
edu=np.copy(train_set['education-num'])
edu_test=np.copy(test_set['education-num'])

def transform5(edu,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if edu[i] <5:
            edu[i] = 0
        elif edu[i]>=5 and edu[i]<10:
            edu[i] = 1
        elif edu[i]>=10:
            edu[i]=2
    return edu
edu=transform5(edu,m)
edu_test=transform5(edu_test,n)

train_set['education-num']=edu
test_set['education-num']=edu_test
```

4.8 将 maritial_status 分为两类：married 和 not-married

```python
#8.将maritial_status 分为两类 married  not-married
print(train_set['marital-status'].value_counts())
mari=np.copy(train_set['marital-status'])
mari_test=np.copy(test_set['marital-status'])
def transform8(mari,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if mari[i]==" Divorced" or mari[i]==" Never-married" or mari[i]==" Separated" or mari[i]==" Widowed":
            mari[i]=" not-married"
        else:
            mari[i]=" married"
    return mari
mari=transform8(mari,m)
mari_test=transform8(mari_test,n)

train_set['marital-status']=mari
test_set['marital-status']=mari_test
```

4.9 将 occupation 分为 High、Med、Low 三类

```python
#9.将occupation分为High  Med  Low三类
occu=np.copy(train_set['occupation'])
occu_test=np.copy(test_set['occupation'])
def transform9(occu,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if occu[i]==" Prof-specialty" or occu[i]==" Exec-managerial":
            occu[i]=" High"
        elif occu[i]==" Tech-support" or occu[i]==" Transport-moving" or \
                occu[i]==" Protective-serv" or occu[i]==" Sales" or occu[i]==" Craft-repair" \
                or occu[i]==" Armed-Forces":
            occu[i]=" Med"
        else:
            occu[i]=" Low"
    return occu
occu=transform9(occu,m)
occu_test=transform9(occu_test,n)

train_set['occupation']=occu
test_set['occupation']=occu_test
```

4.10 将 relationship 分为 Husband、Other、Wife

```python
#10.将relationship 分为Husband  Other   Wife
reship=np.copy(train_set['relationship'])
reship_test=np.copy(test_set['relationship'])
def transform7(reship,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if reship[i]!=" Husband" and reship[i]!=" Wife":
            reship[i] = " Other"
    return reship
reship=transform7(reship,m)
reship_test=transform7(reship_test,n)

train_set['relationship']=reship
test_set['relationship']=reship_test
```

4.11 将 race 分为两类：White 和 Other。

```
#11.将race分为两类  White和Other
race=np.copy(train_set['race'])
race_test=np.copy(test_set['race'])
def transform10(race,size):  #多了个空格的问题  2 用法
    for i in range(size):
        if race[i]!=" White":
            race[i] = " Other"
    return race
race=transform10(race,m)
race_test=transform10(race_test,n)

train_set['race']=race
test_set['race']=race_test
```

5）处理标签列及测试集

```
#对income属性列（即标签列）进行标签编码
label_encoder=LabelEncoder()
train_set['income']=label_encoder.fit_transform(train_set['income'])
test_set['income'] = label_encoder.fit_transform(test_set['income'])

#将测试集转为列表，并将测试集上的标签列存储在y_test中
y_test=test_set['income'].values.tolist()
y_train=train_set['income'].values.tolist()
# train_set = train_set.values.tolist()
test_set = test_set.values.tolist()
```

ID3 决策树

①创建数据集函数

```
# 测试决策树的构建
dataSet,featureName = createDataSet()
```

```
##创建数据集
def createDataSet():  1个用法
    dataSet=train_set.values.tolist()
    featureName=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                 'capital-gain','capital-loss','hours-per-week','native-country','income']
    # 返回数据集和每个维度的名称
    return dataSet, featureName
```

②创建决策树

```
#创建决策树
myTree = createTree(dataSet,featureName)
```

```python
#创建决策树
def createTree(dataSet,featureName,chooseBestFeatureToSplitFunc = chooseBestFeatureToSplitByID3):  3 用法
    """
    创建决策树
    :param dataSet: 数据集
    :param featureName: 数据集每一维的名称
    :return: 决策树
    """
    classList = [example[-1] for example in dataSet] # 类别列表
    if classList.count(classList[0]) == len(classList): # 统计属于列别classList[0]的个数
        return classList[0] # 当类别完全相同则停止继续划分
    if len(dataSet[0]) ==1: # 当只有一个特征的时候，遍历所有实例返回出现次数最多的类别  即特征为类别时
        return majorityCnt(classList) # 返回类别标签
    bestFeat = chooseBestFeatureToSplitFunc(dataSet)#最佳特征对应的索引
    bestFeatLabel = featureName[bestFeat] #最佳特征的名称
    myTree ={bestFeatLabel:{}}  # map 结构，且key为featureLabel
    del (featureName[bestFeat])
    # 找到需要分类（最佳特征）的特征子集
    featValues = [example[bestFeat] for example in dataSet]
    uniqueVals = set(featValues)
    for value in uniqueVals:
        subLabels = featureName[:] # 复制操作 将最佳特征删除后的featureName
        #递归调用createTree函数
        myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet,bestFeat,value),subLabels)
    return myTree
```

其中创建决策树函数的实现流程如下：

1）用 classList 列表来装数据集上的分类列

```python
classList = [example[-1] for example in dataSet] # 类别列表
```

2）判断 classList 列表中装的值是否都相同，如果相同（类别完全相同）则返回该相同值

```python
if classList.count(classList[0]) == len(classList): # 统计属于列别classList[0]的个数
    return classList[0] # 当类别完全相同则停止继续划分
```

3）当只有一个特征的时候，遍历所有实例返回出现次数最多的类别（调用函数 majorityCnt）

```python
if len(dataSet[0]) ==1: # 当只有一个特征的时候，遍历所有实例返回出现次数最多的类别  即特征为类别时
    return majorityCnt(classList) # 返回类别标签
```

```python
#计算classList中出现次数最多的元素
def majorityCnt(classList):  1 个用法
    classCount = {}
    for vote in classList: #统计classList中每个元素出现的次数
        if vote not in classCount.keys():
            classCount[vote] = 0
        classCount[vote] += 1
    sortedClassCount = sorted(classCount.items(), key = operator.itemgetter(1), reverse = True) #根据字典的值降序排序
    return sortedClassCount[0][0]      #返回classList中出现次数最多的元素
```

4）找出决策树最佳划分特征（调用函数 chooseBestFeatureToSplicFunc），构建字典

```python
bestFeat = chooseBestFeatureToSplitFunc(dataSet)#最佳特征对应的索引
bestFeatLabel = featureName[bestFeat] #最佳特征的名称
myTree ={bestFeatLabel:{}}  # map 结构，且key为featureLabel
del (featureName[bestFeat])
```

其中 chooseBestFeatureToSplitByID3 函数又调用了以下函数来计算每个特征的信息增益来比较选出最佳特征

计算信息熵

```python
##计算训练数据集中的Y随机变量的香农熵（信息熵）
def calcShannonEnt(dataSet):  2 用法
    numEntries = len(dataSet) # 实例的个数
    labelCounts = {}
    for featVec in dataSet: # 遍历每个实例，统计标签的频次
        currentLabel = featVec[-1] # 表示最后一列
        # 当前标签不在labelCounts map中，就让labelCounts加入该标签
        if currentLabel not in labelCounts.keys():
            labelCounts[currentLabel] =0
        labelCounts[currentLabel] +=1
    shannonEnt = 0.0
    for key in labelCounts:
        prob = float(labelCounts[key]) / numEntries
        shannonEnt -= prob * log(prob, base: 2) # log base 2
    return shannonEnt
```

计算条件熵

```python
## 计算条件熵
def calcConditionalEntropy(dataSet,i,featList,uniqueVals):  1 个用法
    """
    计算x_i给定的条件下，Y的条件熵
    :param dataSet: 数据集
    :param i: 维度i
    :param featList: 数据集特征列表
    :param uniqueVals: 数据集特征集合  该维度i所有出现的值
    :return: 条件熵
    """
    ce = 0.0
    for value in uniqueVals:
        subDataSet = splitDataSet(dataSet,i,value)
        prob = len(subDataSet) / float(len(dataSet)) # 极大似然估计概率
        ce += prob * calcShannonEnt(subDataSet) #∑pH(Y|X=xi) 条件熵的计算
    return ce
```

计算信息增益

```python
##计算信息增益
def calcInformationGain(dataSet,baseEntropy,i):  1个用法
    """
    :param dataSet: 数据集
    :param baseEntropy: 数据集中Y的信息熵
    :param i: 特征维度i
    :return: 特征i对数据集的信息增益g(dataSet | X_i)
    """
    featList = [example[i] for example in dataSet] # 第i维特征列表
    uniqueVals = set(featList) # 换成集合 - 集合中的每个元素不重复
    newEntropy = calcConditionalEntropy(dataSet,i,featList,uniqueVals)#计算条件熵,
    infoGain = baseEntropy - newEntropy # 信息增益 = 信息熵 - 条件熵
    return infoGain
```

5）用 featValues 列表装该最佳特征列的所有值，uniqueVals 装该列中出现过的值（只记值的种类）

```python
# 找到需要分类（最佳特征）的特征子集
featValues = [example[bestFeat] for example in dataSet]
uniqueVals = set(featValues)
```

6）遍历 uniqueVals，当按照 uniqueVals 中的值 value 划分数据集（调用函数 splitDataSet）之后，再在新的数据集重新寻找最佳特征，递归创建决策树的分支

```python
for value in uniqueVals:
    subLabels = featureName[:] # 复制操作 将最佳特征删除后的featureName
    #递归调用createTree函数
    myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet,bestFeat,value),subLabels)
```

```python
##分割数据集，只取第axis列中值为value的对应样本（行）
def splitDataSet(dataSet,axis,value):  2 用法
    """
    按照给定特征划分数据集
    :param axis:划分数据集的特征的维度   特征维度的名称
    :param value:特征的值
    :return: 符合该特征的所有实例（并且自动移除掉这维特征）
    """
    # 循环遍历dataSet中的每一行数据
    retDataSet = []
    for featVec in dataSet:
        if featVec[axis] == value:
            reduceFeatVec = featVec[:axis] # 删除这一维特征
            reduceFeatVec.extend(featVec[axis+1:])
            retDataSet.append(reduceFeatVec)
    return retDataSet
```

决策树中分支路停止创建的条件为：

新的数据集上的类别列的值都相同时，或者新的数据集上只剩下分类列这一列（无特征列时）

```python
if classList.count(classList[0]) == len(classList): # 统计属于列别classList[0]的个数
    return classList[0] # 当类别完全相同则停止继续划分
```

```python
if len(dataSet[0]) ==1: # 当只有一个特征的时候，遍历所有实例返回出现次数最多的类别   即特征为类别时
    return majorityCnt(classList) # 返回类别标签
```

递归完后决策树创建成功

③根据决策树对测试集上的数据进行分类

```python
featLabels=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                'capital-gain','capital-loss','hours-per-week','native-country','income']
featLabels_train=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                'capital-gain','capital-loss','hours-per-week','native-country','income']
classList=[]      #用来装预测分类结果
classList_train=[]  #用来装训练集上的预测分类结果
for i in range(len(test_set)):
    classLabel = classify(myTree, featLabels, test_set[i])
    classList.append(classLabel)
```

分类函数 classify 定义如下

```python
def classify(inputTree,featLabels,testVec):  4 用法
    #字典中的第一个键firstStr
    firstStr = list(inputTree.keys())[0]
    secondDict = inputTree[firstStr]
    #在featLabels找到该键对应在测试集上的属性列index
    featIndex = featLabels.index(firstStr)
    #测试数据中该属性列的具体值
    key = testVec[featIndex]
    #当该值不在字典中存在时，用该键firstStr的存在的其他值替换该值，用其他存在值的分类结果代替
    if key not in secondDict:
        key=list(secondDict.keys())[0]
    valueOfFeat = secondDict[key]
    if isinstance(valueOfFeat, dict):
        classLabel = classify(valueOfFeat, featLabels, testVec)
    else: classLabel = valueOfFeat
    return classLabel
```

因为是在训练集上训练的决策树，有些测试集中出现的情况可能在训练集里从未出现过，导致用测试集进行测试时，寻找决策树找不到该种情况。比如在训练集中 属性 A 的值为 a,属性 B 的值为 b,c,d 的情况都有出现，但是测试集中出现了 属性 A 的值为 a,属性 B 的值为 f 的情况，这时就决策树就没法对该种情况分类。

处理这种情况的方法本应是返回到上一个属性 A 值为 a 的情况下分类中哪种类别的占比大，就用把该条数据判为哪类。但是因为在函数都构建好后才发现会出现这个问题，修改起来很复杂，这里就简单的将该种不知情况的分类结果改为属性 A 值为 a，属性 B 值为 b 的情况下的分类了。

④计算准确率和 AUC 值

```python
#计算训练集上的准确率
correct_num=0
for i in range(n):
    # y_test[i].replace('.',"")
    # print(y_test[i])
    if(classList[i]==y_test[i]):
        correct_num+=1

rate=correct_num/n
print('决策树准确率: ',rate)
```

```python
#计算AUC值
from sklearn.metrics import roc_auc_score
auc_score=roc_auc_score(y_test,classList)
print('决策树AUC值: ',auc_score)
```

随机森林实现详细步骤:

①定义参数

```
n_features = 8   #特征属性个数
n_trees = 10     #子决策树个数
```

②根据样本集及 n_trees 参数构建随机森林

```
RandomForest(dataSet,n_trees)
# print(len(TreeList))
```

```
#构建随机森林，用TreeList来存储
TreeList=[]
def RandomForest(dataSet,n_trees):  1个用法
    for i in range(n_trees):
        final_subData,subFeature=generateDataSet(dataSet,featLabels,n_features)
        myTree=createTree(final_subData,subFeature)
        print(myTree)
        TreeList.append(myTree)
    return TreeList
```

每一个子决策树的构建步骤如下：

1）从样本集中随机构造出的数据子集

```
# 随机构造数据子集
def get_subsample(dataSet):  1个用法
    subdataSet = []
    lenSubdata = len(dataSet)
    while len(subdataSet) < lenSubdata:
        index = randrange(len(dataSet) - 1)
        subdataSet.append(dataSet[index])
    return subdataSet
```

2）随机选中 n_features 个特征

```
#随机n_features个特征
def get_subfeature(featLabels,n_features):  1个用法
    subFeature=[]
    subFeatIndex=random.sample(range(0,len(featLabels)-1),n_features)
    for i in range(len(subFeatIndex)):
        subFeature.append(featLabels[subFeatIndex[i]])
    subFeature.append('income')
    subFeatIndex.append(12)
    return subFeature,subFeatIndex
```

3）生成构建该子决策树的数据子集

```python
#从数据子集中选出随机的n_features，构成最终进行决策树构建的数据集
def generateDataSet(dataSet,featLabels,n_features):  1个用法
    subdataSet=get_subsample(dataSet)
    subFeature,subFeatIndex=get_subfeature(featLabels,n_features)
    print(subFeature,subFeatIndex)
    final_subData=[]
    for i in range(len(subdataSet)):
        row_list = []
        for j in range(n_features+1):
            row_list.append(subdataSet[i][subFeatIndex[j]])
        final_subData.append(row_list)
    return final_subData,subFeature
```

4）利用数据子集和 createTree 函数生成子决策树

```python
myTree=createTree(final_subData,subFeature)
```

这样循环 n_trees 次构建了 n_trees 个子决策树，将所有子决策树都存放在 TreeList 列表中。

③根据随机森林计算每条测试集数据在每个子决策树上的分类情况

```python
#predList用来装每条测试集上的数据在各个子决策树上的分类结果，列表中的每个元素代表某条数据在各个子决策树上的分类结果
predList=[]
def cal(TreeList):  1个用法
    for j in range(len(test_set)):
        #classList1用来装某条数据集在各个子决策树上的分类结果
        classList1=[]
        for i in range(len(TreeList)):
            #调用classify函数判断在子决策树上的分类
            classLabel1 = classify(TreeList[i], featLabels, test_set[j])
            # print(classLabel,i)
            classList1.append(classLabel1)
        predList.append(classList1)
cal(TreeList)
# print(len(predList))
```

④投票决定每条数据的最终分类结果（调用了 Counter 函数用来记票数）

```python
#true_pred用来装每条数据的最终分类结果
true_pred=[]
collection_words_list=[]
for i in range(len(test_set)):
    #调用Counter 计算某条数据在各子决策树的分类结果中，票数统计情况，按照票数从高到低排列分类结果
    collection_words_list.append(Counter(predList[i]))
    #取第一个分类结果，即票数最高的分类结果
    true_pred.append(list(collection_words_list[i].keys())[0])
```

⑤计算准确率和 AUC 值

```
#计算准确率
correct_num1=0
for i in range(n):
    if(true_pred[i]==y_test[i]):
        correct_num1+=1

rate1=correct_num1/n
print('随机森林准确率为: ',rate1)

#计算AUC值
auc_score1=roc_auc_score(y_test,true_pred)
print('随机森林AUC值: ',auc_score1)
```

（3）展示实验结果

```
随机森林准确率为:  0.8281540504648074
随机森林AUC值:  0.7254451370384468
```

（4）讨论实验结果，分析贪心决策树、随机森林分类的优劣

①由准确率及 AUC 值可以看出，在 adult 数据集上对测试集进行分类，贪心决策树算法反而比随机森林分类表现得更好，但是由以往对贪心决策树和随机森林分类算法的讨论，随机森林分类应该是要比贪心决策树的性能高。

此时再计算一下决策树在 adult 训练数据集上的分类准确值，重新运行程序得到结果：由训练集和测试集上的准确率差别（差距不大）可知，决策树算法在该数据集上不算出现过拟合现象

```
决策树准确率:  0.8243691899070386
决策树训练集上准确率:  0.8503083349910483
决策树AUC值:  0.7448965074229159
```

其中随机森林算法中对参数特征属性个数以及子决策树的指定个数确实会影响到算法的分类结果，但以上得出的随机森林准确率已经是不断调整两个参数的值后，以及随机子数据集和特征后得到的较好结果。

②前人得出的贪心决策树和随机森林算法的优劣

贪心决策树：

优点：好理解，可支持非线性问题的解决，不需要数据归一化。可以用于特征工程，特征向量的选择。

缺点：容易出现过拟合，微小的数据改变会改变整个树的形状。

随机森林：

优点：减少过拟合情况，预测值不会因为训练数据的大小变化而剧烈变化。

③可能正是由于在该数据集上决策树算法没有出现过拟合现象，而随机森林本就是在决策树的基础上建成并且能够减少过拟合情况，所以随机森林和贪心决策树算法的准确率没有相差很大，甚至随机森林的性能还要差些（扩展：可以给出相应的改进措施）。

（5）源代码

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from math import log
```

```
import operator
import random
from random import randrange
from collections import Counter

#导入数据集
train_set=pd.read_csv('oldData/adult.data',header=None,names=['age','workclass','fnlwgt','education',

'education-num','marital-status',

'occupation','relationship','race',

'sex','capital-gain','capital-loss',

'hours-per-week','native-country','income'])
train_set.info()
print(train_set.describe())
test_set=pd.read_csv('oldData/adult.test',header=None,names=['age','workclass','fnlwgt','education',
                                                                                    'education-num','marital-status',
                                                                                    'occupation','relationship','race',
                                                                                    'sex','capital-gain','capital-loss',

'hours-per-week','native-country','income'])
test_set.info()
# print(test_set)


#处理缺失值 将含有值' ? '的数据行去掉  注意问号前有一个空格, 处理缺失值后数据的 index 变成不是连续的
for i in train_set.columns:
    test_set=test_set[test_set[i]!=' ?']
    train_set=train_set[train_set[i]!=' ?']

#重新规划数据集的 index
test_set.index=range(0,len(test_set))
train_set.index=range(0,len(train_set))
# print(train_set)
# print(test_set)

#删除 fnlgwt 列,序号列
train_set.drop('fnlwgt',axis=1,inplace=True)
test_set.drop('fnlwgt',axis=1,inplace=True)

#Eductaion 和 EduNum 特征相似，可以删除 Education
```

```
train_set.drop(['education'],axis=1,inplace=True)
test_set.drop(['education'],axis=1,inplace=True)

#m,n 分别为训练集和测试集的行数
m=train_set.shape[0]
n=test_set.shape[0]

#1.将 age 属性划分为 0-25    25-50      50-75      75-100
age=np.copy(train_set['age'])
age_test=np.copy(test_set['age'])

def transform1(age,size):
        for i in range(size):
                if age[i]>=0 and age[i]<25:
                        age[i]=0
                elif age[i]>=25 and age[i]<50:
                        age[i]=1
                elif age[i]>=50 and age[i]<75:
                        age[i]=2
                elif age[i]>=75:
                        age[i]=3
        return age
age=transform1(age,m)
age_test=transform1(age_test,n)
train_set['age']=age
test_set['age']=age_test


#2.将 capital-gain 属性       >0 的值都用 1 替换      =0 的值都用 0 替换
gain=np.copy(train_set['capital-gain'])
gain_test=np.copy(test_set['capital-gain'])

def transform2(gain,size):
        for i in range(size):
                if gain[i]==0:
                        gain[i]=0
                elif gain[i]>0:
                        gain[i]=1
        return gain

gain=transform2(gain,m)
gain_test=transform2(gain_test,n)
train_set['capital-gain']=gain
test_set['capital-gain']=gain_test
```

```
# print(train_set['capital-gain'])


#3.将 captional-loss 属性  划分为>0      =0 两类
loss=np.copy(train_set['capital-loss'])
loss_test=np.copy(test_set['capital-loss'])

loss=transform2(loss,m)
loss_test=transform1(loss_test,n)
train_set['capital-loss']=loss
test_set['capital-loss']=loss_test

# print(train_set['capital-loss'])

#4.将 housr-per-week 划分为          <40      ==40      >40
hours=np.copy(train_set['hours-per-week'])
hours_test=np.copy(test_set['hours-per-week'])

def transform3(hours,size):
    for i in range(size):
        if hours[i] < 40:
            hours[i] = 0
        elif hours[i] == 40:
            hours[i] = 1
        elif hours[i] >40:
            hours[i]=2
    return hours
hours=transform3(hours,m)
hours_test=transform3(hours_test,n)

train_set['hours-per-week']=hours
test_set['hours-per-week']=hours_test

# print(train_set['hours-per-week'])

#5.将 Country 划分为 USA    not USA 两类
cty=np.copy(train_set['native-country'])
cty_test=np.copy(test_set['native-country'])

def transform4(cty,size):    #多了个空格的问题
    for i in range(size):
        if cty[i] ==" United-States":
            cty[i] = 0
```

```
        elif cty[i] != " United-States":
            cty[i] = 1
    return cty
cty=transform4(cty,m)
cty_test=transform4(cty_test,n)

train_set['native-country']=cty
test_set['native-country']=cty_test

# print(train_set['native-country'])
#
# print(train_set.info())
# print(train_set)

#6.将 workclass 分为 Freelance    other    Proprietor    Private   Government 五类
# print(train_set['workclass'].value_counts())
workclass=np.copy(train_set['workclass'])
workclass_test=np.copy(test_set['workclass'])
def transform6(workclass,size):   #多了个空格的问题
    for i in range(size):
        if workclass[i]==" Federal-gov" or workclass[i]== " Local-gov" or workclass[i]== " State-gov" :
            workclass[i] = " Government"
        elif workclass[i] == " Self-emp-not-inc" or   workclass[i]==" Self-emp-inc":
            workclass[i] = " Proprietor"
    return workclass
cty=transform6(workclass,m)
cty_test=transform6(workclass_test,n)

train_set['workclass']=workclass
test_set['workclass']=workclass_test

# print(train_set['workclass'])

# print(train_set.info())
# print(train_set)
# print(train_set['workclass'].value_counts())

#7.将 education-num 分为 0-5:0    5-10:1   >=10:2
edu=np.copy(train_set['education-num'])
edu_test=np.copy(test_set['education-num'])

def transform5(edu,size):   #多了个空格的问题
    for i in range(size):
        if edu[i] <5:
```

```
            edu[i] = 0
        elif edu[i]>=5 and edu[i]<10:
            edu[i] = 1
        elif edu[i]>=10:
            edu[i]=2
    return edu
edu=transform5(edu,m)
edu_test=transform5(edu_test,n)

train_set['education-num']=edu
test_set['education-num']=edu_test
#
# print(train_set['education-num'])

#8.将 maritial_status 分为两类 married   not-married
print(train_set['marital-status'].value_counts())
mari=np.copy(train_set['marital-status'])
mari_test=np.copy(test_set['marital-status'])
def transform8(mari,size):   #多了个空格的问题
    for i in range(size):
        if mari[i]==" Divorced" or mari[i]==" Never-married" or mari[i]==" Separated" or mari[i]==" Widowed":
            mari[i]=" not-married"
        else:
            mari[i]=" married"
    return mari
mari=transform8(mari,m)
mari_test=transform8(mari_test,n)

train_set['marital-status']=mari
test_set['marital-status']=mari_test

#9.将 occupation 分为 High   Med   Low 三类
occu=np.copy(train_set['occupation'])
occu_test=np.copy(test_set['occupation'])
def transform9(occu,size):   #多了个空格的问题
    for i in range(size):
        if occu[i]==" Prof-specialty" or occu[i]==" Exec-managerial":
            occu[i]=" High"
        elif occu[i]==" Tech-support" or occu[i]==" Transport-moving" or \
                occu[i]==" Protective-serv" or occu[i]==" Sales" or occu[i]==" Craft-repair" \
                or occu[i]==" Armed-Forces":
            occu[i]=" Med"
        else:
```

```
                occu[i]=" Low"
        return occu
occu=transform9(occu,m)
occu_test=transform9(occu_test,n)

train_set['occupation']=occu
test_set['occupation']=occu_test

# print(train_set.info())
# print(train_set)
# print(train_set['occupation'].value_counts())

#10.将 relationship  分为 Husband    Other      Wife
reship=np.copy(train_set['relationship'])
reship_test=np.copy(test_set['relationship'])
def transform7(reship,size):    #多了个空格的问题
        for i in range(size):
                if reship[i]!=" Husband" and reship[i]!=" Wife":
                        reship[i] = " Other"
        return reship
reship=transform7(reship,m)
reship_test=transform7(reship_test,n)

train_set['relationship']=reship
test_set['relationship']=reship_test

# print(train_set.info())
# print(train_set)
# print(train_set['relationship'].value_counts())

#11.将 race 分为两类    White 和 Other
race=np.copy(train_set['race'])
race_test=np.copy(test_set['race'])
def transform10(race,size):    #多了个空格的问题
        for i in range(size):
                if race[i]!=" White":
                        race[i] = " Other"
        return race
race=transform10(race,m)
race_test=transform10(race_test,n)

train_set['race']=race
test_set['race']=race_test
```

```
#对 income 属性列（即标签列）进行标签编码
label_encoder=LabelEncoder()
train_set['income']=label_encoder.fit_transform(train_set['income'])
test_set['income'] = label_encoder.fit_transform(test_set['income'])

#将测试集转为列表，并将测试集上的标签列存储在 y_test 中
y_test=test_set['income'].values.tolist()
y_train=train_set['income'].values.tolist()
# train_set = train_set.values.tolist()
test_set = test_set.values.tolist()
#-----------数据预处理完毕


#-----------------------决策树-----------------------
#如果我把这些数据都转变为 0,1,2，那么如果拿到一条新的测试数据，我就得先将它转成离散值的形式，然后再进行决策树算法的判断

##创建数据集
def createDataSet():
    dataSet=train_set.values.tolist()
    featureName=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                 'capital-gain','capital-loss','hours-per-week','native-country','income']
    # 返回数据集和每个维度的名称
    return dataSet, featureName

##分割数据集
def splitDataSet(dataSet,axis,value):
    """
    按照给定特征划分数据集
    :param axis:划分数据集的特征的维度　特征维度的名称
    :param value:特征的值
    :return: 符合该特征的所有实例（并且自动移除掉这维特征）
    """
    # 循环遍历 dataSet 中的每一行数据
    retDataSet = []
    for featVec in dataSet:
        if featVec[axis] == value:
            reduceFeatVec = featVec[:axis] # 删除这一维特征
            reduceFeatVec.extend(featVec[axis+1:])
            retDataSet.append(reduceFeatVec)
    return retDataSet

##计算训练数据集中的 Y 随机变量的香农熵（信息熵）
def calcShannonEnt(dataSet):
```

```
    numEntries = len(dataSet) #  实例的个数
    labelCounts = {}
    for featVec in dataSet: #  遍历每个实例，统计标签的频次
        currentLabel = featVec[-1] #  表示最后一列
        #  当前标签不在 labelCounts map 中，就让 labelCounts 加入该标签
        if currentLabel not in labelCounts.keys():
            labelCounts[currentLabel] =0
        labelCounts[currentLabel] +=1
    shannonEnt = 0.0
    for key in labelCounts:
        prob = float(labelCounts[key]) / numEntries
        shannonEnt -= prob * log(prob,2) # log base 2
    return shannonEnt

##  计算条件熵
def calcConditionalEntropy(dataSet,i,featList,uniqueVals):
    """
    计算 x_i 给定的条件下，Y 的条件熵
    :param dataSet:  数据集
    :param i:  维度 i
    :param featList:  数据集特征列表
    :param unqiueVals:  数据集特征集合    该维度 i 所有出现的值
    :return:  条件熵
    """
    ce = 0.0
    for value in uniqueVals:
        subDataSet = splitDataSet(dataSet,i,value)
        prob = len(subDataSet) / float(len(dataSet)) #  极大似然估计概率
        ce += prob * calcShannonEnt(subDataSet) #∑pH(Y|X=xi)  条件熵的计算
    return ce

##计算信息增益
def calcInformationGain(dataSet,baseEntropy,i):
    """
    :param dataSet:  数据集
    :param baseEntropy:  数据集中 Y 的信息熵
    :param i:  特征维度 i
    :return:  特征 i 对数据集的信息增益 g(dataSet | X_i)
    """
    featList = [example[i] for example in dataSet] #  第 i 维特征列表
    uniqueVals = set(featList) #  换成集合 - 集合中的每个元素不重复
    newEntropy = calcConditionalEntropy(dataSet,i,featList,uniqueVals)#计算条件熵,
    infoGain = baseEntropy - newEntropy #  信息增益 = 信息熵 - 条件熵
    return infoGain
```

```
## 选择最好的数据特征划分，返回最佳特征对应的维度 index
def chooseBestFeatureToSplitByID3(dataSet):
    numFeatures = len(dataSet[0]) -1 # 最后一列是分类　特征维度的数量
    baseEntropy = calcShannonEnt(dataSet) #返回整个数据集的信息熵
    bestInfoGain = 0.0
    bestFeature = -2     #用来记录信息增益最大的特征的索引值,注意不要用-1,若是 bestInfoGain 一直没有变化，进行划分的就是分类列了
    for i in range(numFeatures): # 遍历所有维度特征
        infoGain = calcInformationGain(dataSet,baseEntropy,i) #返回具体特征的信息增益
        # print(infoGain)
        if(infoGain > bestInfoGain):
            bestInfoGain = infoGain
            bestFeature = i
    return bestFeature # 返回最佳特征对应的维度

#计算 classList 中出现次数最多的元素
def majorityCnt(classList):
    classCount = {}
    for vote in classList: #统计 classList 中每个元素出现的次数
        if vote not in classCount.keys():
            classCount[vote] = 0
        classCount[vote] += 1
    sortedClassCount = sorted(classCount.items(), key = operator.itemgetter(1), reverse = True) #根据字典的值降序排序
    return sortedClassCount[0][0]          #返回 classList 中出现次数最多的元素

#创建决策树
def createTree(dataSet,featureName,chooseBestFeatureToSplitFunc = chooseBestFeatureToSplitByID3):
    """
    创建决策树
    :param dataSet: 数据集
    :param featureName: 数据集每一维的名称
    :return: 决策树
    """
    classList = [example[-1] for example in dataSet] # 类别列表
    if classList.count(classList[0]) == len(classList): # 统计属于列别 classList[0]的个数
        return classList[0] # 当类别完全相同则停止继续划分
    if len(dataSet[0]) ==1: # 当只有一个特征的时候，遍历所有实例返回出现次数最多的类别　即特征为类别时
        return majorityCnt(classList) # 返回类别标签
    bestFeat = chooseBestFeatureToSplitFunc(dataSet)#最佳特征对应的索引
    bestFeatLabel = featureName[bestFeat] #最佳特征的名称
    myTree ={bestFeatLabel:{}}   # map 结构，且 key 为 featureLabel
```

```
    del (featureName[bestFeat])
    # 找到需要分类（最佳特征）的特征子集
    featValues = [example[bestFeat] for example in dataSet]
    uniqueVals = set(featValues)
    for value in uniqueVals:
        subLabels = featureName[:] # 复制操作 将最佳特征删除后的 featureName
        #递归调用 createTree 函数
        myTree[bestFeatLabel][value] = createTree(splitDataSet(dataSet,bestFeat,value),subLabels)
    return myTree

# 测试决策树的构建
dataSet,featureName = createDataSet()
#创建决策树
myTree = createTree(dataSet,featureName)
# print(myTree)
# print(test_set[0])


def classify(inputTree,featLabels,testVec):
    #字典中的第一个键 firstStr
    firstStr = list(inputTree.keys())[0]
    secondDict = inputTree[firstStr]
    #在 featLabels 找到该键对应在测试集上的属性列 index
    featIndex = featLabels.index(firstStr)
    #测试数据中该属性列的具体值
    key = testVec[featIndex]
    #当该值不在字典中存在时，用该键 firstStr 的存在的其他值替换该值，用其他存在值的分类结果代替
    if key not in secondDict:
        key=list(secondDict.keys())[0]
    valueOfFeat = secondDict[key]
    if isinstance(valueOfFeat, dict):
        classLabel = classify(valueOfFeat, featLabels, testVec)
    else: classLabel = valueOfFeat
    return classLabel

featLabels=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                    'capital-gain','capital-loss','hours-per-week','native-country','income']
featLabels_train=['age','workclass','education-num','marital-status','occupation','relationship','race','sex',
                    'capital-gain','capital-loss','hours-per-week','native-country','income']
classList=[]        #用来装预测分类结果
classList_train=[]  #用来装训练集上的预测分类结果
for i in range(len(test_set)):
    classLabel = classify(myTree, featLabels, test_set[i])
```

```
    classList.append(classLabel)

for i in range(len(dataSet)):
    classLabel_train = classify(myTree, featLabels_train, dataSet[i])
    classList_train.append(classLabel_train)
#计算训练集上的准确率
correct_num=0
for i in range(n):
    # y_test[i].replace('.',"")
    # print(y_test[i])
    if(classList[i]==y_test[i]):
        correct_num+=1

rate=correct_num/n
print('决策树准确率：',rate)

#计算准确率
correct_num_train=0
for i in range(m):
    if(classList_train[i]==y_train[i]):
        correct_num_train+=1

rate_train=correct_num_train/m
print('决策树训练集上准确率：',rate_train)

#计算 AUC 值
from sklearn.metrics import roc_auc_score
auc_score=roc_auc_score(y_test,classList)
print('决策树 AUC 值：',auc_score)


#----------------------------随机森林的创建------------------------
n_features = 8    #特征属性个数
n_trees = 10       #子决策树个数

#  随机构造数据子集
def get_subsample(dataSet):
    subdataSet = []
    lenSubdata = len(dataSet)
    while len(subdataSet) < lenSubdata:
        index = randrange(len(dataSet) - 1)
        subdataSet.append(dataSet[index])
    return subdataSet
```

```
#随机 n_features 个特征
def get_subfeature(featLabels,n_features):
    subFeature=[]
    subFeatIndex=random.sample(range(0,len(featLabels)-1),n_features)
    for i in range(len(subFeatIndex)):
        subFeature.append(featLabels[subFeatIndex[i]])
    subFeature.append('income')
    subFeatIndex.append(12)
    return subFeature,subFeatIndex

#从数据子集中选出随机的 n_features，构成最终进行决策树构建的数据集
def generateDataSet(dataSet,featLabels,n_features):
    subdataSet=get_subsample(dataSet)
    subFeature,subFeatIndex=get_subfeature(featLabels,n_features)
    print(subFeature,subFeatIndex)
    final_subData=[]
    for i in range(len(subdataSet)):
        row_list = []
        for j in range(n_features+1):
            row_list.append(subdataSet[i][subFeatIndex[j]])
        final_subData.append(row_list)
    return final_subData,subFeature

#构建随机森林，用 TreeList 来存储
TreeList=[]
def RandomForest(dataSet,n_trees):
    for i in range(n_trees):
        final_subData,subFeature=generateDataSet(dataSet,featLabels,n_features)
        myTree=createTree(final_subData,subFeature)
        print(myTree)
        TreeList.append(myTree)
    return TreeList

RandomForest(dataSet,n_trees)
# print(len(TreeList))

#predList 用来装每条测试集上的数据在各个子决策树上的分类结果，列表中的每个元素代表某条数据在各个子决策树上的分类结果
predList=[]
def cal(TreeList):
    for j in range(len(test_set)):
        #classList1 用来装某条数据集在各个子决策树上的分类结果
        classList1=[]
        for i in range(len(TreeList)):
```

```
                #调用 classify 函数判断在子决策树上的分类
                classLabel1 = classify(TreeList[i], featLabels, test_set[j])
                # print(classLabel,i)
                classList1.append(classLabel1)
            predList.append(classList1)
cal(TreeList)
# print(len(predList))

#true_pred 用来装每条数据的最终分类结果
true_pred=[]
collection_words_list=[]
for i in range(len(test_set)):
    #调用 Counter 计算某条数据在各子决策树的分类结果中，票数统计情况，按照票数从高到低排列分类结果
    collection_words_list.append(Counter(predList[i]))
    #取第一个分类结果，即票数最高的分类结果
    true_pred.append(list(collection_words_list[i].keys())[0])

# print(true_pred)

#计算准确率
correct_num1=0
for i in range(n):
    if(true_pred[i]==y_test[i]):
        correct_num1+=1

rate1=correct_num1/n
print('随机森林准确率为：',rate1)

#计算 AUC 值
auc_score1=roc_auc_score(y_test,true_pred)
print('随机森林 AUC 值：',auc_score1)
```